Models in Data Science — Lab Exam Quick Answers

SET 1

1. NumPy arrays from Python data structures, intrinsic objects and random functions

```
import numpy as np

a = np.array([1, 2, 3])       # from list
b = np.array((4, 5, 6))       # from tuple
z = np.zeros((2, 3))
o = np.ones((2, 2))
r = np.arange(1, 10, 2)
l = np.linspace(0, 1, 5)
x = np.random.randint(1, 10, (2, 3))

print(a, b, z, o, r, l, x, sep='\\n')
```

2. Indexing, slicing, reshaping, joining and splitting

```
import numpy as np

a = np.arange(1, 7)
print(a[2])                # indexing
print(a[1:4])              # slicing
b = a.reshape(2, 3)        # reshape
print(b)
print(np.concatenate((a, a))) # joining
print(np.split(a, 2))      # splitting into equal parts
```

3. Universal functions and mathematical methods

```
import numpy as np

a = np.array([1, 4, 9])
print(np.sqrt(a))
print(np.square(a))
print(np.sum(a), np.mean(a), np.median(a))
print(np.min(a), np.max(a), np.std(a))
print(np.add(a, 2), np.multiply(a, 3))
```

4. CSV statistical and comparison operations

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
print(df.describe())              # statistics for numeric columns
print(df.mean(numeric_only=True))
print(df.max(numeric_only=True))
print(df["Age"] > 25)             # comparison; change Age to a numeric column
print(df[df["Age"] > 25])         # matching rows
```

5. Load an image, crop and flip using NumPy indexing

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

img = np.array(Image.open("image.jpg"))
crop = img[50:200, 50:200]       # crop rows and columns
flip = img[:, ::-1]              # horizontal flip

plt.subplot(1, 3, 1); plt.imshow(img); plt.title("Original")
plt.subplot(1, 3, 2); plt.imshow(crop); plt.title("Crop")
```

```
plt.subplot(1, 3, 3); plt.imshow(flip); plt.title("Flipped")
plt.show()
```

6. Different color scales on rainfall data

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Example rainfall values; replace with rainfall CSV data if provided
df = pd.DataFrame({"Jan":[20, 35, 15], "Feb":[30, 25, 40],
                   "Mar":[45, 20, 32]}, index=["City A", "City B", "City C"])
sns.heatmap(df, annot=True, cmap="Blues")
plt.title("Rainfall - Blues")
plt.show()
sns.heatmap(df, annot=True, cmap="YlGnBu")
plt.title("Rainfall - YlGnBu")
plt.show()
sns.heatmap(df, annot=True, cmap="coolwarm")
plt.title("Rainfall - coolwarm")
plt.show()
```

SET 2

1. Read CSV, first/last 10 rows, shape/index/columns, select and delete

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head(10))
print(df.tail(10))
print("Shape:", df.shape)
print("Index:", df.index)
print("Columns:", df.columns)
print(df[df["Age"] > 25])       # select rows by condition
df = df[df["Age"] > 25]         # keep only matching rows
df = df.drop(columns=["Name"])  # delete a column
print(df)
```

2. Different bar plots

```
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv(“iris.csv”)

df.groupby("species")["sepal_length"].mean().plot(kind="bar")
plt.title("Vertical Bar Graph")
plt.ylabel("Sepal Length")
plt.show()

df.groupby("species")["sepal_length"].mean().plot(kind="barh")
plt.title("Horizontal Bar Graph")
plt.xlabel("Sepal Length")
plt.show()

df.groupby("species")[["sepal_length", "petal_length"]].mean().plot(kind="bar")
plt.title("Clustered Bar Graph")
plt.ylabel("Length")
plt.show()
```

SET 3

1. Ranking, sorting, statistics, category counts and renaming

```
import pandas as pd

df = pd.read_csv("data.csv")
df["Rank"] = df["Marks"].rank(ascending=False)
print(df.sort_values("Marks", ascending=False))
print(df["Marks"].describe())
print(df["Category"].value_counts())
print(df["Category"].unique())
df = df.rename(columns={"Marks":"Score", "Category":"Group"})
print(df.head())
```

2. Decision Tree Classifier with Gini Index on categorical data

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier

df = pd.DataFrame({
    "Outlook":["Sunny","Sunny","Overcast","Rain","Rain","Rain","Overcast","Sunny"],
    "Wind":["Weak","Strong","Weak","Weak","Strong","Weak","Strong","Weak"],
    "Play":["No","No","Yes","Yes","No","Yes","Yes","No"]
})
for c in df.columns:
    df[c] = LabelEncoder().fit_transform(df[c])

X = df[["Outlook", "Wind"]]
y = df["Play"]
model = DecisionTreeClassifier(criterion="gini", random_state=0)
model.fit(X, y)
print("Training accuracy:", model.score(X, y))
print("Prediction:", model.predict([[1, 1]]))
```

SET 4

1. Residual analysis of simple linear regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 6])
model = LinearRegression().fit(X, y)
pred = model.predict(X)
residual = y - pred
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Residuals:", residual)
```

2. Residual plot of linear regression

```
import matplotlib.pyplot as plt

plt.scatter(pred, residual)
plt.axhline(0, color="red")
plt.xlabel("Predicted values")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()
```

3. KNN classifier on Iris dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

SET 5

1. Normal probability (Q-Q) plot

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

data = np.array([12, 15, 14, 16, 18, 19, 20, 22, 24, 25])
stats.probplot(data, dist="norm", plot=plt)
plt.title("Normal Probability Plot")
plt.show()
```

2. Empirical model of linear regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 3, 5, 4, 6])
model = LinearRegression().fit(X, y)
print("Empirical model: y =", model.coef_[0], "* x +", model.intercept_)
print("Predicted y for x=6:", model.predict([[6]])[0])
```

3. SVC with linear kernel on Iris dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = SVC(kernel="linear")
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

SET 6

1. Detect/fill/drop missing values; apply() and map()

```
import pandas as pd

df = pd.DataFrame({"Name":["A","B","C","D"],
                   "Age":[20, None, 22, 25],
                   "Grade":["a","b","a","c"]})
print(df.isnull().sum())            # detect missing values
```

```
print(df.dropna())                  # drop rows with missing values
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Age"] = df["Age"].apply(lambda x: x + 1)
df["Grade"] = df["Grade"].map({"a":1, "b":2, "c":3})
print(df)
```

2. Histogram

```
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("iris.csv")
plt.hist(df["sepal_length"], bins=10, edgecolor="black")
plt.xlabel("Sepal length"); plt.ylabel("Frequency")
plt.title("Histogram"); plt.show()
```

SET 7

1. Detect and filter outliers using IQR

```
import pandas as pd

df = pd.DataFrame({"Marks":[35, 40, 42, 45, 48, 50, 52, 200]})
Q1 = df["Marks"].quantile(0.25)
Q3 = df["Marks"].quantile(0.75)


IQR = Q3 - Q1
low = Q1 - 1.5 * IQR
high = Q3 + 1.5 * IQR

outliers = df[(df["Marks"] < low) | (df["Marks"] > high)]
clean = df[(df["Marks"] >= low) & (df["Marks"] <= high)]
print("Outliers:\n", outliers)
print("Filtered data:\n", clean)
```

2. KNN with 5-fold cross-validation on Iris

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
model = KNeighborsClassifier(n_neighbors=30)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv)
print("Fold accuracies:", scores)
print("Mean accuracy:", scores.mean())
```

SET 8

1. CSV details, selection/deletion, ranking and sorting

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head(10)); print(df.tail(10))
print(df.shape); print(df.index); print(df.columns)

print(df[df["Marks"] > 50])          # select by condition
df = df.drop(columns=["Name"])       # delete column
df["Rank"] = df["Marks"].rank(ascending=False)
print(df.sort_values("Marks", ascending=False))
```

2. Line plot

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data.csv")
plt.plot(df["hours"], df["marks"], marker="o")
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.title("Hours vs Marks")
plt.show()
```

SET 9

1. Statistics, categorical counts/uniqueness and rename columns

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.describe())
print(df["Category"].value_counts())
print(df["Category"].unique())
df = df.rename(columns={"Marks":"Score", "Category":"Group"})
print(df.head())
```

2. Scatter plot

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(“iris.csv”)
sns.scatterplot(data=df, x="sepal_length", y="petal_length",
                hue="species")
plt.title("Sepal Length vs Petal Length")
plt.show()
```